

Lab5 用户程序

一、练习1: 加载应用程序并执行

1.代码补充说明

代码块

```
1    tf->gpr.sp = USTACKTOP;
```

将tf->gpr.sp设置为用户栈的顶部地址，当程序返回用户模式后，它需要知道从哪里开始使用和管理它的栈空间，这个地址就是用户程序自己的栈指针。

代码块

```
1    tf->epc = elf->e_entry;
```

将tf->epc设置为用户程序的入口点，在RISC-V架构中，当从内核返回时，CPU会跳转到sepc寄存器中保存的地址。因此，tf->epc对应的值就是用户程序的入口地址，来确保程序从正确的位置开始执行。

代码块

```
1    tf->status = (sstatus & ~SSTATUS_SPP & ~SSTATUS_SIE) | SSTATUS_SPIE;
```

tf->status应该设置为适用于用户程序的模式和权限状态，从而控制CPU的运行模式和权限状态。在RISC-V中，这对应的是sstatus。

2.用户态进程被选择到指令执行经过

(1) 调度选择阶段

- `schedule()` 函数遍历进程链表，查找状态为 `PROC_RUNNABLE` 的进程
- 找到可运行进程后，调用 `proc_run(next)` 开始切换

2.进程切换

`proc_run()` 函数执行三个关键操作：

- 更新 **current** 指针： `current = proc`

- **切换页表**：如果新旧进程页表不同，修改 `satp` 寄存器指向新进程的页表
- **上下文切换**：调用 `switch_to(&prev->context, &next->context)`

3.上下文切换

`switch_to`汇编函数

- 保存当前进程的 `ra` , `sp` , `s0-s11` 寄存器到 `prev->context`
- 从 `next->context` 恢复新进程的寄存器
- `ret` 指令会跳转到 `ra` 指向的 `forkret`

4.进入forkret

代码块

```
1  static void forkret(void) {
2      forkrets(current->tf);
3  }
```

调用 `forkrets` , 参数是当前进程的 `trapframe` 地址

5.执行forkrets

代码块

```
1  forkrets:
2      move sp, a0          # 将栈指针设为 trapframe 地址
3      j __trapret         # 跳转到陷阱返回代码
```

6.陷阱返回__trapret

代码块

```
1  __trapret:
2      RESTORE_ALL          # 恢复所有寄存器
3      sret                 # 特权级返回指令
```

`RESTORE_ALL` 宏从 `trapframe` 中恢复：

- 所有通用寄存器 `x0-x31` （包括用户栈指针 `sp`）
- `sepc` 寄存器（保存用户程序的入口地址）
- 将用户栈指针恢复到 `sscratch`

7.sret指令执行

- CPU 从 `sepc` 寄存器读取用户程序入口地址
- 根据 `sstatus` 切换到用户态 (U-mode)
- PC 跳转到用户程序第一条指令开始执行

二、练习2: 父进程复制自己的内存空间给子进程

1. 原始 `copy_range` 函数设计实现过程

pmm.c

```
1  int copy_range(pde_t *to, pde_t *from, uintptr_t start, uintptr_t end, bool
    share)
2  {
3      assert(start % PGSIZE == 0 && end % PGSIZE == 0);
4      assert(USER_ACCESS(start, end));
5      // copy content by page unit.
6      do
7      {
8          // call get_pte to find process A's pte according to the addr start
9          pte_t *ptep = get_pte(from, start, 0), *nptep;
10         if (ptep == NULL)
11         {
12             start = ROUNDDOWN(start + PTSIZE, PTSIZE);
13             continue;
14         }
15         // call get_pte to find process B's pte according to the addr start. If
16         // pte is NULL, just alloc a PT
17         if (*ptep & PTE_V)
18         {
19             if ((nptep = get_pte(to, start, 1)) == NULL)
20             {
21                 return -E_NO_MEM;
22             }
23             uint32_t perm = (*ptep & PTE_USER);
24             // get page from ptep
25             struct Page *page = pte2page(*ptep);
26             int ret = 0;
27             assert(page != NULL);
28             // alloc a page for process B
29             struct Page *npage = alloc_page();
30             assert(npage != NULL);
31             // (1) 源页的内核虚拟地址
32             void *src_kvaddr = page2kva(page);
```

```

33         // (2) 目标页的内核虚拟地址
34         void *dst_kvaddr = page2kva(npage);
35         // (3) 拷贝一页内容
36         memcpy(dst_kvaddr, src_kvaddr, PGSIZE);
37         // (4) 在 to (子进程) 的页表中建立线性地址 start 的映射
38         ret = page_insert(to, npage, start, perm);
39         assert(ret == 0);
40     }
41     start += PGSIZE;
42 } while (start != 0 && start < end);
43 return 0;
44 }
```

- `void *src_kvaddr = page2kva(page);`
 - 将父进程页表里指向的源物理页 `page` 转成内核虚拟地址。
- `void *dst_kvaddr = page2kva(npage);`
 - 将为子进程新分配的物理页 `npage` 转成对应的内核虚拟地址。
 - 同样返回该页在内核地址空间中的线性映射起始地址。
- `memcpy(dst_kvaddr, src_kvaddr, PGSIZE);`
 - 以页为单位拷贝数据：从父进程源页拷贝整整一页到新页。
- `ret = page_insert(to, npage, start, perm);`
 - 在子进程的页表 `to` 中，把线性地址 `start` 映射到新物理页 `npage`，权限为从父页复制来的 `perm`。

2. 设计实现Copy on Write (COW) 机制

在 `fork()` 时不立即复制用户页，改为父子共享物理页并设为只读；第一次写发生时才复制，保证写者看到独立副本，读者保持共享以节省内存与加速fork。

使用页表权限（移除写位PTE_W使为只读）+物理页引用计数（`page->ref`）来判断是否需要复制；通过页故障捕获写请求并完成复制与页表更新。

三、练习3: 阅读分析源代码，理解进程执行fork/exec/wait/exit 的实现，以及系统调用的实现

1. fork / exec / wait / exit 的实现

前半部分以 `fork` 为例（其他三个过程几乎一致）：

- 用户程序调用 `fork()` 函数（`user/xxx.c`中），`fork()` 函数定义在`user/libs/ulib.[h|c]`;

- `fork()` 调用 `sys_fork()` 函数，`sys_fork()` 函数定义在 `user/libs/syscall.[h|c]`;
- `sys_fork()` 函数调用 `syscall(SYS_fork)`，`syscall()` 函数定义在 `user/libs/syscall.c`;
- `syscall()` 函数内部有内联汇编，把系统调用号和参数放到寄存器 `a0~a5`，然后执行 `ecall`;
- `ecall` 生成异常，被异常处理入口捕获，最后进入 `kern/trap/trap.c` 的 `exception_handler()` 函数;
- 对用户态 `CAUSE_USER_ECALL`：调用内核的 `syscall()`（定义在 `kern/syscall/syscall.[h|c]` 中）（**进入内核**）;
- 内核的 `syscall()` 从当前进程的 `trapframe` 拿到系统调用号和参数，调用 `sys_fork()`（定义在 `kern/syscall/syscall.c` 中），并把 `sys_fork()` 的返回值写回 `tf->gpr.a0`;
- `sys_fork()` 调用 `do_fork()`（定义在 `kern/process/proc.c` 中）。

后半部分：

- `do_fork`：在 `proc.c` 中，
 - 在内核分配并初始化子进程描述符（`alloc_proc`、`setup_kstack`）。
 - 复制或共享地址空间（`copy_mm`：若非 `CLONE_VM`，会创建新的 `mm` 并通过 `dup_mmap`/复制页表拷贝用户空间；`pmm` 中的 `copy_range/pgdir_alloc_page` 参与物理页分配与复制，见 `pmm.c`）。
 - 设置子进程的 `trapframe/context`（`copy_thread`），并明确设置子进程 `tf->gpr.a0 = 0`（使子进程 `fork` 返回值为 0）。
 - 将子进程标为可运行并唤醒（`wakeup_proc`）；父进程的 `sys_fork` 调用最终返回子 `pid`（由 `syscall()` 写入父进程的 `trapframe.a0`）。
 - 切回用户态前：内核返回值已经放到当前进程的 `trapframe.a0`，返回到用户态时寄存器 `a0` 恢复该值。
- `do_execve`：在 `proc.c` 的 `do_execve / load_icode`：
 - 验证并释放（或切换）当前进程的旧 `mm`（切回内核页表后释放旧页表），然后为新 ELF 建立新的 `mm`、页表、复制段、建立用户栈等。
 - 设置当前进程的 `current->mm`、`pgdir`，并写入 `trapframe`：`tf->gpr.sp`（用户栈）、`tf->epc`（程序入口）和 `tf->status`（用户态 `sstatus`）。
 - 当内核返回到用户态时，进程不会继续原来代码，而从新的 ELF `entry` 开始执行（因此 `exec` 在用户态表现为“原进程被新程序替换”）。
- `do_wait`：在 `proc.c` 的 `do_wait`：
 - 检查用户传入的 `code_store` 指针合法性（`user_mem_check`）。

- 遍历子进程查找已成为 PROC_ZOMBIE 的子进程；若找到则把子退出码复制到用户提供的位置（通过内核访问用户地址），并释放子进程的内核资源（unhash/remove_links/put_kstack/kfree）。
- 如果没有可收集的僵尸子进程但存在子进程，则把当前进程置为 PROC_SLEEPING、设置 wait_state = WT_CHILD，然后 schedule()（阻塞当前进程，让出 CPU，内核切换到别的进程）。
- 当子进程退出时（见 do_exit），内核会 wakeup_proc(parent) 把父进程标记为可运行；调度器会在合适时刻恢复父进程，父进程再次执行 do_wait 并完成返回工作。
- `do_exit`：在 proc.c 的 do_exit：
 - 释放或减引用当前进程的 mm（lsatp 切回内核页表以安全释放用户页表），设置 current->mm = NULL。
 - 将进程状态置为 PROC_ZOMBIE，保存 exit_code。
 - 把子进程改为 initproc 的子），并在必要时唤醒 initproc 或父进程（如果父进程在等待，wakeup_proc(parent) 会将父进程从睡眠唤醒）。
 - 最后调用 schedule() 切出当前进程（do_exit 不会返回）。

内核/用户如何交错执行：

用户态代码运行 → 执行 ecall → CPU 进入内核（trap），内核在内核态完成系统调用的全部必要工作。如果内核在处理时调用 `schedule()` 阻塞当前进程（例如 `do_wait` 或 `do_exit` 后），内核会上下文切换到其他进程（可能是内核线程或其他用户进程）。当对应事件发生（子进程 exit 等）内核会 `wakeup_proc` 使被阻塞进程变为 RUNNABLE，调度器把它切回来继续执行。恢复回用户态时，trap 返回路径会把 trapframe 中的寄存器（包括 a0）恢复到用户寄存器并执行 `sret` 返回用户态。

内核结果如何传回用户程序：

内核通过修改当前进程的 trapframe 来传递返回值与上下文，当 trap 返回并恢复寄存器后，用户态看到的函数返回值正是寄存器 a0 的值（即内核写入的值）。

2. 用户态进程的执行状态生命周期图

代码块

```

1      PROC_UNINIT ← -----do_wait()-----+
2      |                                         |
3      | wakeup_proc()                         |
4      ↓                                         |
5      PROC_RUNNABLE ----- do_exit() -----+ |
6      ↑                                         | |
7      |                                         | |
8      | ↓do_wait()                             | |
9      | ↑wakeup()                             | |

```

```

10      |                               ↓   |
11      +---→ PROC_SLEEPING -----→ PROC_ZOMBIE

```

四、扩展练习 Challenge 1

1. 实现源码

1.1 用COW改写 `copy_range`（替换pmm.c中原有`copy_range`实现）

pmm.c

```

1  int copy_range(pde_t *to, pde_t *from, uintptr_t start, uintptr_t end, bool
    share)
2  {
3      assert(start % PGSIZE == 0 && end % PGSIZE == 0);
4      assert(USER_ACCESS(start, end));
5      do
6      {
7          pte_t *ptep = get_pte(from, start, 0), *nptep;
8          if (ptep == NULL)
9          {
10             start = ROUNDDOWN(start + PTSIZE, PTSIZE);
11             continue;
12          }
13          if (*ptep & PTE_V)
14          {
15             if ((nptep = get_pte(to, start, 1)) == NULL)
16             {
17                 return -E_NO_MEM;
18             }
19             uint32_t perm = (*ptep & PTE_USER);
20             struct Page *page = pte2page(*ptep);
21             assert(page != NULL);
22
23             // Set up COW: make both mappings read-only (clear write bit)
24             uint32_t roperm = perm & (~PTE_W);
25             // increment refcount
26             page_ref_inc(page);
27             // update child pte to point to same page, read-only
28             *nptep = pte_create(page2ppn(page), roperm);
29             // update parent pte to read-only if it was writable
30             *ptep = pte_create(page2ppn(page), roperm);
31             // invalidate TLB entries for both page tables (at least for
    parent)
32             tlb_invalidate(from, start);

```

```

33         tlb_invalidate(to, start);
34     }
35     start += PGSIZE;
36 } while (start != 0 && start < end);
37 return 0;
38 }

```

1.2 在vmm.h添加页故障处理函数声明，在vmm.c实现页故障处理（处理写时复制）

vmm.h

```

1 int do_pgfault(struct mm_struct *mm, uint32_t error_code, uintptr_t addr);

```

vmm.c

```

1 #include <pmm.h>
2 #include <error.h>
3
4 int do_pgfault(struct mm_struct *mm, uint32_t error_code, uintptr_t addr)
5 {
6     if (mm == NULL)
7     {
8         return -E_INVALID;
9     }
10
11     uintptr_t la = ROUNDDOWN(addr, PGSIZE);
12     // get pte in this mm's pgdir
13     pte_t *ptep = get_pte(mm->pgdir, la, 0);
14     if (ptep == NULL || !(*ptep & PTE_V))
15     {
16         return -E_INVALID;
17     }
18
19     // error_code: 0 -> load, 1 -> store (we'll only handle store/write here)
20     if (!error_code)
21     {
22         // load fault on a present page: should be permission issue
23         return -E_INVALID;
24     }
25
26     struct Page *page = pte2page(*ptep);
27     if (page == NULL)
28     {
29         return -E_INVALID;

```



```

30     }
31
32     // If shared (refcount > 1) => allocate new page, copy, update pte
33     if (page_ref(page) > 1)
34     {
35         struct Page *npage = alloc_page();
36         if (npage == NULL)
37         {
38             return -E_NO_MEM;
39         }
40         memcpy(page2kva(npage), page2kva(page), PGSIZE);
41         page_ref_dec(page);
42         // set pte to new page with write permission
43         uint32_t perm = (*ptep & PTE_USER) | PTE_W;
44         *ptep = pte_create(page2ppn(npage), perm);
45         tlb_invalidate(mm->pgdir, la);
46     }
47     else
48     {
49         // sole owner: just make writable
50         uint32_t perm = (*ptep & PTE_USER) | PTE_W;
51         *ptep = pte_create(page2ppn(page), perm);
52         tlb_invalidate(mm->pgdir, la);
53     }
54     return 0;
55 }

```

1.3 在页错误分支调用do_pgfault (trap.c的页故障case)

```

trap.c
1  case CAUSE_LOAD_PAGE_FAULT:
2      if (current != NULL && !trap_in_kernel(tf))
3      {
4          if (do_pgfault(current->mm, 0, tf->tval) == 0)
5              return;
6          // fallthrough: unhandled -> kill
7      }
8      cprintf("Load page fault\n");
9      break;
10 case CAUSE_STORE_PAGE_FAULT:
11     if (current != NULL && !trap_in_kernel(tf))
12     {
13         if (do_pgfault(current->mm, 1, tf->tval) == 0)
14             return;
15         // unhandled -> kill

```

```
16     }
17     cprintf("Store/AMO page fault\n");
18     break;
```

2. 测试用例（全都放在user目录下）

2.1 cow_basic.c 基本COW验证 (子改父不变)

```
cow_basic.c

1  #include <ulib.h>
2  #include <stdio.h>
3
4  static char buf[4096]; // one page
5
6  int main(void) {
7      buf[0] = 1;
8      int pid = fork();
9      if (pid == 0) {
10         buf[0] = 2;
11         cprintf("child: buf[0]=%d\n", buf[0]);
12         exit(0);
13     } else {
14         wait();
15         cprintf("parent: buf[0]=%d\n", buf[0]);
16     }
17     return 0;
18 }
```

2.2 cow_parent_first.c 父先写后子读

```
cow_parent_first.c

1  #include <ulib.h>
2  #include <stdio.h>
3
4  static char buf[4096];
5
6  int main(void) {
7      buf[0] = 10;
8      int pid = fork();
9      if (pid == 0) {
10         for (volatile int i=0;i<100000;i++); // let parent run first
11         cprintf("child reads: buf[0]=%d\n", buf[0]);
12         exit(0);
```

```

13     } else {
14         buf[0] = 20;
15         cprintf("parent wrote buf[0]=%d\n", buf[0]);
16         wait();
17     }
18     return 0;
19 }

```

2.3 cow_multi_pages.c 两页分别修改验证按页复制

```

cow_multi_pages.c

1  #include <ulib.h>
2  #include <stdio.h>
3
4  static char a[4096];
5  static char b[4096];
6
7  int main(void) {
8      a[0] = 1; b[0] = 2;
9      int pid = fork();
10     if (pid == 0) {
11         a[0] = 11;
12         cprintf("child: a=%d b=%d\n", a[0], b[0]);
13         exit(0);
14     } else {
15         b[0] = 22;
16         wait();
17         cprintf("parent: a=%d b=%d\n", a[0], b[0]);
18     }
19     return 0;
20 }

```

2.4 cow_many_children.c 多子进程各自改不同页

```

cow_many_children.c

1  #include <ulib.h>
2  #include <stdio.h>
3
4  #define NCH 4
5  static char bigbuf[4096 * NCH];
6
7  int main(void) {
8      for (int i=0; i<NCH; i++) bigbuf[i*4096] = i;

```

```

9     for (int i=0;i<NCH;i++) {
10         int pid = fork();
11         if (pid == 0) {
12             bigbuf[i*4096] = i + 100;
13             cprintf("child %d: buf[%d]=%d\n", i, i, bigbuf[i*4096]);
14             exit(0);
15         }
16     }
17     for (int i=0;i<NCH;i++) wait();
18     for (int i=0;i<NCH;i++) cprintf("parent: buf[%d]=%d\n", i, bigbuf[i*4096]);
19     return 0;
20 }

```

测试方法

- 添加进CMakeLists.txt文件

```

113     user/yield.c
114     user/cow_basic.c
115     ..... user/cow_parent_first.c
116     ..... user/cow_multi_pages.c
117     ..... user/cow_many_children.c)
118

```

- 修改kern/process/proc.c user_name()

```

proc.c

1     static int
2     user_main(void *arg)
3     {
4     #ifdef TEST
5         KERNEL_EXECVE2(TEST, TESTSTART, TESTSIZE);
6     #else
7         KERNEL_EXECVE(exit);
8         // 改为以下依次测试
9         // KERNEL_EXECVE(cow_basic);
10        // KERNEL_EXECVE(cow_parent_first);
11        // KERNEL_EXECVE(cow_multi_pages);
12        // KERNEL_EXECVE(cow_many_children);
13    #endif
14        panic("user_main execve failed.\n");
15    }

```

- 运行 `make qemu`

```
kernel_execve: pid = 2, name = "cow_basic".
Breakpoint
child: buf[0]=2
parent: buf[0]=1
all user-mode processes have quit.
init check memory pass.
```

cow_basic.c

```
kernel_execve: pid = 2, name = "cow_parent_first".
Breakpoint
parent wrote buf[0]=20
child reads: buf[0]=20
all user-mode processes have quit.
init check memory pass.
```

cow_parent_first

```
kernel_execve: pid = 2, name = "cow_multi_pages".
Breakpoint
child: a=11 b=22
parent: a=1 b=22
all user-mode processes have quit.
init check memory pass.
```

cow_multi_pages

```
kernel_execve: pid = 2, name = "cow_many_children".
Breakpoint
child 3: buf[3]=103
child 2: buf[2]=102
child 1: buf[1]=101
child 0: buf[0]=100
parent: buf[0]=0
parent: buf[1]=1
parent: buf[2]=2
parent: buf[3]=3
all user-mode processes have quit.
init check memory pass.
```

cow_many_children

3. 设计报告

3.1 核心数据与不变式

- 物理页描述： `struct Page` 包含 `ref`（引用计数）。
- 页表项语义：一个用户可见的私有页在共享时应为只读（`PTE_W`清除）；写权限恢复只在本进程独占页（`page->ref == 1`）或在写时复制到新页后。
- 不变式：对任意用户页，
 - 如果有N个有效PTE指向该物理页，则 `page->ref == N`。
 - 只要 `page->ref > 1`，所有指向它的PTE上的写位必须被清除（只读），从而保证写操作触发页故障。
 - 修改PTE时必须调用 `tlb_invalidate()`。

3.2 各种状态

- State A — Exclusive Writable
 - 描述： `page->ref == 1`，且对应进程的PTE可写（`PTE_W = 1`）。
 - 来源：新分配页；独占后（写者已获得写权限）。
 - 允许行为：直接写入；无需复制。
 - 触发转移：`fork()`（父在复制范围内）：把父PTE改为只读，增加 `page->ref` → 转入 State B（Shared Read-Only）。
- State B — Shared Read-Only (COW shared)

- 描述: `page->ref >= 2` , 所有指向该页的用户PTE的写位为 0。
- 来源: `copy_range` 在`fork()`时把父/子PTE都设置为只读并 `page_ref_inc()` 。
- 允许行为: 读 (不复制) ; 任何写触发 `STORE_PAGE_FAULT` (页错) 。
- 触发转移: 某进程发生写错 -> 进入复制处理 (State C) , 见下。
- State C — Copying (transient)
 - 描述: 页错误处理正在为写方分配并复制到新页。
 - 来源: `do_pgfault` 检测 `page_ref > 1` 并分配新页。
 - 允许行为: 不对外暴露; 完成后写方的PTE指向新页 (ref为1, 可写) , 原页 `ref--` 。
 - 触发转移: 成功复制 -> 写方进入State A (独占可写) ; 失败 -> 返回错误/杀进程。
- State D — Shared Writable (not allowed for COW private pages)
 - 描述: `page->ref >= 2` 且至少存在一个PTE有写位。
 - 说明: 在正确的COW实现中此状态应被禁止 (写位必须在共享时清除) 。若出现, 表明同步/实现错误。应通过修复PTE或触发复制回退到安全状态。

五、扩展练习 Challenge 2

1. 何时被预先加载: 在编译/链接阶段被打包进内核镜像 (linker 生成符号 `_binary_obj__user_<name>_out_start/_size`) , 因此当内核被引导载入内存时这些用户程序的二进制数据已经在内存里了。
2. 实际运行时流程: `KERNEL_EXECVE` 宏把嵌入的二进制指针传给 `kernel_execve` , 内核最终在 `do_execve` → `load_icode` 中把嵌入的ELF节拷贝并映射到新进程的用户地址空间。
3. 与常见操作系统的区别: 常见OS (有文件系统) 是在exec/load时从磁盘/文件系统读取ELF并加载到进程内存; 本系统没有在执行时从磁盘读取, 而是把所有用户程序作为二进制blob静态链接进内核镜像, 启动时就驻留在内存中。
4. 原因: 教学/实验用的ucore系统为了简单性和可重复性 (没有完整文件系统/存储子系统)、方便在模拟器/裸机上运行与评分, 采用静态嵌入的方式; 同时内核仍通过`load_icode`做ELF解析并按进程建立独立的用户地址空间 (即把嵌入数据复制到进程页表中) 。

六、分支任务: Lab2 gdb调试页表查询过程

在本次的调试过程中, 我们需要打开三个终端进行调试:

1.终端1: 启动QEMU模拟器

首先我们在其中一个终端中输入以下命令来启动QEMU模拟器:

代码块

```
1 make debug
```

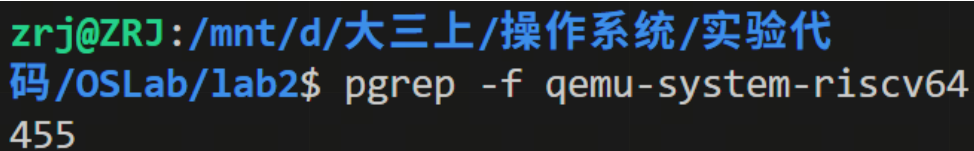
2.终端2：附加调试QEMU进程

然后我们在第二个终端（之后我们称为终端A）中依次输入以下命令

代码块

```
1 pgrep -f qemu-system-riscv64
```

该命令是查询正在运行的qemu-system-riscv64进程的进程ID（PID），然后系统返回对应进程的ID:



```
zrj@ZRJ:/mnt/d/大三上/操作系统/实验代码/OSLab/lab2$ pgrep -f qemu-system-riscv64
455
```

然后我们执行以下的命令，进入QEMU的调试的阶段：

代码块

```
1 sudo gdb
2 attatch 455
3 handle SIGPIPE nostop noprint
```

其中 `handle SIGPIPE nostop noprint` 这一句命令的作用是告诉调试器忽略掉某种特定的干扰信号，从而保证调试过程不被莫名其妙地中断，同时也能保证终端页面的干净，GDB只会在真正触发设置的断点时才停下来。

```
(gdb) attach 455
Attaching to process 455
[New LWP 456]
[New LWP 457]
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libt
hread_db.so.1".
0x000079708e918d3e in __ppoll (fds=0x5c120f234890, nfd
s=7,
timeout=<optimized out>, sigmask=0x0) at ../sysdeps/unix/sy
sv/linux/ppoll.c:42
42      ../sysdeps/unix/sysv/linux/ppoll.c: No such file or
directory.
(gdb) handle SIGPIPE nostop noprint
Signal          Stop      Print    Pass to program      Descri
ption
SIGPIPE         No       No       Yes Broken pipe
```

QEMU进入调试阶段之后，我们先在QEMU中设置无条件断点。

代码块

```
1  # 典型通用入口：TLB miss
2  break tlb_fill
3  # RISC-V 专属页表行走（常见）
4  break riscv_cpu_tlb_fill
5  continue
```

在 `tlb_fill` 这个C函数入口处设置一个断点，`tlb_fill` 是QEMU软件TLB的通用管理函数，当CPU访问内存没命中缓存，就一定会经过这里，在该处设置一个断点可以拦截所有发生的 `TLB Miss`，从而详细的查看当 `TLB Miss` 后操作系统是如何查询页表以及将对应的内容填充到TLB中。

然后在 `riscv_cpu_tlb_fill` 处设置一个断点，这是RISC-V架构特有的TLB填充函数，也就是说当通用 `tlb_fill` 被调用后，它会进一步调用这个RISC-V专属函数。此时，通过观察诸如`satp`寄存器状态，确认当前是否开启了分页模式以及页表节点的物理地址；同时，结合 `mmu_idx` 和 `access_type`，我们可以判断当前 CPU 是在 Sv32、Sv39 还是 Sv48 模式下进行地址拆解，并深入追踪代码如何一层层读取物理内存中的页表项（PTE），直到找到最终的物理页框号（PPN）或触发页面异常。

最后输入 `continue` 让虚拟机运行。

3.终端3：调试ucore内核

在QEMU中设置好操作系统的断点之后，我们在ucore处进入调试模式，从而能够跨越硬件模拟与软件逻辑的边界，实时监控ucore内核在初始化页表时对satp寄存器的修改。通过GDB捕获tlb_fill的异常现场，我们可以对照内核源码中物理内存布局与虚拟地址空间的映射关系，验证多级页表项的权限位是否正确配置，进而排查导致内核崩溃或非法访问的根源问题。

代码块

```
1 make gdb
```

我们在第三个终端（之后我们称为终端B）中输入该命令，实现自动化启动GDB客户端，并建立与已经运行的QEMU调试后端的通信连接。建立通信完成后，我们需要将符号文件传入该终端。

完成以上操作后，我们就正式的进入了调试的步骤。

调试过程

然后我们在ucore内核的“总入口函数”处设置一个断点，在进入kern_init之前，内核会设置页表，因此，我们在这里设置断点，我们可以一步一步跟踪到它开启MMU的那一刻。

然后我们输入c，让QEMU开始跑，然后我们观察终端A是否会在打下的断点处停止并输出内容。

```
Thread 3 "qemu-system-ris" hit Breakpoint 1, tlb_fill (cpu
=0x560defe1a510, addr=2147522064, size=4, access_type=MMU_
DATA_LOAD, mmu_idx=3, retaddr=139332329411179) at /home/zr
j/src/qemu-4.1.1/accel/tcg/cputlb.c:871
871          CPUClass *cc = CPU_GET_CLASS(cpu);
```

在终端A中输出了图中的内容，根据图中的内容可知，命中了tlb_fill的断点，当前停在该函数内，根据具体的参数显示，这是一次取指访问，也就是access_type参数的取值所显示的内容：

MMU_INST_FETCH，虚拟地址为4096，mmu_idx=3显示的是MMU权限级别，3表示当前是处在S Mode，说明内核代码在触发转换。总的来说，QEMU的第3号线程在tlb_fill函数处停下，也就是说，RISC-V虚拟机正准备从地址0x1000处取第一条指令，但是QEMU的软件TLB里还没有这个地址的记录，接下来我们查看上下文与参数名来对页表查询进行详细得分析，我们输入 `list` 命令，来查看当前断点附近的源代码，具体的输出如下所示：

代码块

```
1 (gdb) list
2 866      * be discarded and looked up again (e.g. via tlb_entry()).
3 867      */
4 868      static void tlb_fill(CPUState *cpu, target_ulong addr, int size,
5 869                          MMUAccessType access_type, int mmu_idx, uintptr_t
      retaddr)
```

```

6   870   {
7   871       CPUClass *cc = CPU_GET_CLASS(cpu);
8   872       bool ok;
9   873
10  874       /*
11  875         * This is not a probe, so only valid return is success; failure
12  876         * should result in exception + longjmp to the cpu loop.
13  877         */
14  878       ok = cc->tlb_fill(cpu, addr, size, access_type, mmu_idx, false,
15                        retaddr);
16  879       assert(ok);
17  880   }
18  881
19  882   static uint64_t io_readx(CPUArchState *env, CPUIOTLBEntry *iotlbentry,

```

这一段代码显示了QEMU处理内存地址转换失败的枢纽，以下是我们详细分析源代码的具体内容：

代码块

```

1   CPUClass *cc = CPU_GET_CLASS(cpu);

```

这是QEMU面向对象设计的体现。QEMU模拟多种架构，每种架构查找页表的方式都不同，这一行是动态获取当前处理器的“类”，以便后面能调用该架构特有的查表函数。下一句的注释说明该函数无法填充TLB，它不会简单地返回一个错误码，而是会直接跳转去处理异常，也就是缺页异常，不会再回到正常的指令执行流，tlb_fill函数是查询页表的软件实现入口。

代码块

```

1   ok = cc->tlb_fill(cpu, addr, size, access_type, mmu_idx, false, retaddr);

```

这一段显示的是CPU架构专属的TLB填充方法，尝试完成虚拟地址->物理地址的转换并填充TLB。false参数表示“不是第二次重试”。

代码块

```

1   assert(ok);

```

这一句的作用是强制校验，用assert强制要求cc->tlb_fill的返回值ok必须为true（也就是TLB填充成功）。如果ok为false，QEMU会直接崩溃并打印断言失败信息。总而言之，这段代码的核心是强制保证TLB填充的“成功闭环”。

然后我们输入 `info args` 命令，来查看对应的寄存器的内容。

```
(gdb) info args
cpu = 0x560defe1a510
addr = 2147522064
size = 4
access_type = MMU_DATA_LOAD
mmu_idx = 3
retaddr = 139332329411179
```

根据图中的内容可知，这一部分内容与上述的断点处输出的内容是一致的。

接着，我们再输入 `info locals` 指令，来打印当前函数内部定义的所有局部变量及其当前数值。

```
(gdb) info locals
cc = 0x5c120f1f0f20
__func__ = "tlb_fill"
ok = false
__PRETTY_FUNCTION__ = "tlb_fill"
(gdb) █
```

变量cc的值为0x5c120f1f0f20，这是指向RISC-V CPU类实现的指针。QEMU已经成功通过CPU_GET_CLASS(cpu)找到了这台模拟器的“说明书”。接下来代码会通过这个指针去调用RISC-V专用的页表查询函数，ok是一个布尔值，是用于记录地址转换是否成功，因为程序刚开始运行到函数开头，还没有开始真正的“走页表”操作，因此此时为false。

然后我们输入 `next` 指令，进行下一步操作，我们以此输入命令：`set step-mode on` 和 `step` 进入RISC-V专属页表行走函数，结果如下图所示：

```
(gdb) next
878      ok = cc->tlb_fill(cpu, addr, size, access_type, mmu_idx, false,
retaddr);
(gdb) set step-mode on
(gdb) step

Thread 3 "qemu-system-ris" hit Breakpoint 2, riscv_cpu_tlb_fill (cs=0x5c120f
1e8510, address=4096, size=0, access_type=MMU_INST_FETCH, mmu_idx=3, probe=f
alse, retaddr=0) at /home/zrj/src/qemu-4.1.1/target/riscv/cpu_helper.c:438
438      {
```

我们对输出的内容进行分析得：

```
代码块 ok = cc->tlb_fill(cpu, addr, size, access_type, mmu_idx, false, retaddr);
```

命令next对应的输出表示它是在填充一个TLB条目。

代码块

```
1 Thread 3 "qemu-system-ris" hit Breakpoint 2, riscv_cpu_tlb_fill
  (cs=0x5c120f1e8510, address=4096, size=0, access_type=MMU_INST_FETCH,
  mmu_idx=3, probe=false, retaddr=0) at /home/zrj/src/qemu-
  4.1.1/target/riscv/cpu_helper.c:438
2 438 {
```

Thread3说明QEMU正在用第三个线程运行指令，我们之前在riscv_cpu_tlb_fill函数上设了断点，我们执行step进入这个函数，GDB刚好触发了这个断点，在此处停止。at
/home/zrj/src/qemu-4.1.1/target/riscv/cpu_helper.c 这一句显示目前我们处在的位置，该文件夹是QEMU源码中专门负责RISC-V辅助功能的文件，在
target/riscv/cpu_helper.c 中，接下来程序会执行Page Table Walk。

通过查询内存地址，我们在地址0x560dc1748f5b处打下一个断点，

```
(gdb) break *0x560dc1748f5b
Breakpoint 6 at 0x560dc1748f5b: file /home/zrj/src/qemu-4.1.1/target/riscv/cpu_helper.c,
line 158.
(gdb) █
```

在cpu的物理地址查询的函数处打下一个断点：

```
(gdb) break cpu_physical_memory_read
Note: breakpoint 3 also set at pc 0x560dc166b558.
Note: breakpoint 3 also set at pc 0x560dc18a3571.
Note: breakpoint 3 also set at pc 0x560dc16baaab.
Note: breakpoint 3 also set at pc 0x560dc16656c4.
Breakpoint 7 at 0x560dc16656c4: cpu_physical_memory_read. (4 locations)
(gdb) █
```

然后在地址0x560defe231a8处设置一个硬件断点。

```
(gdb) watch *(unsigned long *)0x560defe231a8
Hardware watchpoint 9: *(unsigned long *)0x560defe231a8
(gdb) █
```

之后我们就一直输入continue来观察在到达断点时的状态即可。

在上面的步骤中已经显示了我们到达了tlb_fill函数处以及riscv_cpu_tlb_fill函数处，之后我们输入continue到达get_physical_address函数处，也就是实现物理地址解析。

```
(gdb) continue
Continuing.

Thread 3 "qemu-system-ris" hit Breakpoint 6, get_physical_address (env=0x94d7feec9f791800
, physical=0x8, prot=0x560defd6fb30, addr=94617085313163, access_type=22029, mmu_idx=-104
4217768) at /home/zrj/src/qemu-4.1.1/target/riscv/cpu_helper.c:158
158      {
(gdb)
```

riscv_cpu_tlb_fill函数调用get_physical_address函数，我们查看env.satp的值为0,也就是说明为直接映射。在物理地址解析得到PPN后，tlb_set_page_with_attrs计算要写入的tag和value,value包含物理页基址PPN与标志位，在写入neg.tlb之前，QEMU会先加自旋锁，也就是使用qemu_spin_lock函数实现：

```
(gdb) continue
Continuing.

Thread 3 "qemu-system-ris" hit Hardware watchpoint 10: *(unsigned long *)0x560defe227d0

Old value = 34359738368
New value = 34359738369
0x0000560dc168d030 in qemu_spin_lock (spin=0x560defe227d0) at /home/zrj/src/qemu-4.1.1/
include/qemu/thread.h:201
201      while (unlikely(__sync_lock_test_and_set(&spin->value, true))) {
(gdb)
```

然后我们输入finish指令，观察输出的值：

代码块

```
1  (gdb) finish
2  Run till exit from #0  0x0000560dc168d030 in qemu_spin_lock
   (spin=0x560defe227d0) at /home/zrj/src/qemu-4.1.1/include/qemu/thread.h:201
3  tlb_set_page_with_attrs (cpu=0x560defe1a510, vaddr=2147586048,
   paddr=2147586048, attrs=..., prot=7, mmu_idx=3, size=4096) at
   /home/zrj/src/qemu-4.1.1/accel/tcg/cputlb.c:768
4  768          tlb->c.dirty |= 1 << mmu_idx;
```

这一段内容显示QEMU已经成功完成了“找页表”的任务，现在正在正式将转换结果写入其软件TLB缓存中，简单来说，模拟器已经查到了\$0x80010000\$这个地址对应的物理地址，并准备让CPU之后能够“快步通行”。

其中的核心动作为：`tlb_set_page_with_attrs`。现在我们来对其参数进行详细的分析：

- **vaddr=2147586048** (\$0x80010000\$)：这是虚拟地址。在RISC-V的ucore实验中，这通常是内核代码段的起始地址。
- **paddr=2147586048** (\$0x80010000\$)：这是物理地址。由于两个值相等，与上文中的env.satp的值为0对应。

- `prot=7`：这是页面权限标志位。
 - $7=1 \text{ (Read)} + 2 \text{ (Write)} + 4 \text{ (Execute)}$
 - 这意味着该页面是可读、可写、可执行的，这符合内核代码段在启动初期的特征
- `mmu_idx=3`：代表当前的内存管理索引。在 QEMU RISC-V 中，`3` 通常对应 **Machine Mode (机器模式)**。

其中最后一行代码为 `tlb->c.dirty |= 1 << mmu_idx;`；这一行代码是在更新 QEMU 内部维护的 TLB 状态位，其含义如下：

- `tlb->c.dirty`：这是一个位图，记录了哪些 MMU 模式的 TLB 缓存已被修改或包含“脏”数
- `1 << mmu_idx`：将第 3 位置为 1。

根据上述的信息可知，我们向 Machine Mode 的缓存里填了一条新规则时，必须标记该模式为“dirty”。这告诉 QEMU 的执行引擎：这个模式已经更新了。下次 TCG（编译器）生成代码访问这个地址时，必须去检查这个新填入的条目，而不是走旧的路径。

然后我们输入 continue 命令，输出结果如下：

代码块

```
1 Thread 3 "qemu-system-ris" hit Hardware watchpoint 10: *(unsigned long
  *)0x560defe227d0
2
3 Old value = 34359738369
4 New value = 34359738368
5 qemu_spin_unlock (spin=0x560defe227d0) at /home/zrj/src/qemu-
  4.1.1/include/qemu/thread.h:221
6 221 }
```

这一段内容标志着 TLB 填充操作的正式结束。这一段的核心动作为释放自旋锁，也就是 `qemu_spin_unlock` 函数的功能，说明关键任务已经完成并且将权限交还，观察点的数值发生了变化，也就是 Old value 到 New value 的变化。

至此，我们已经完成了页表的查询以及 TLB 的填充。

以下是该过程的流程：

代码块

```
1 开始
2 |
3 v
4 [Guest 指令(Load/Store) 执行 - TCG helper]
5 |
6 v
7 [检查软件 TLB neg.tlb 是否命中 vaddr?]
```



```

8  |-- 是 --> [使用 TLB 条目完成访存] --> 结束
9  |
10 `-- 否 --> [调用 tlb_fill / riscv_cpu_tlb_fill]
11 |
12 v
13 [调用 get_physical_address(env, &pa, &prot, address, ...)]
14 |
15 v
16 /-----
17 | env.satp == 0 ? (分页未启) |
18 | 是 -> 直接映射 paddr = vaddr |
19 | 否 -> 进行页表 walk (Sv39 等): |
20 | - 逐级用 cpu_physical_memory_read 读 PTE |
21 | - 验证 PTE (V/R/W/X/PPN) |
22 -----/
23 |
24 v
25 [构造要写入的 tag (qword0) 和 value (qword1)]
26 |
27 v
28 [tlb_set_page_with_attrs 开始写入]
29 |
30 v
31 [qemu_spin_lock(&slot) —(watchpoint 可能首先触发锁写)]
32 |
33 v
34 [写入两条 store 指令: mov %rax,%rcx (tag)]
35 [ mov %rdx,0x8(%rcx) (value)]
36 |
37 v
38 [qemu_spin_unlock(&slot)]
39 |
40 v
41 [TLB 条目已安装, 后续访问可命中 neg.tlb]
42 |
43 v
44 结束

```

七、分支任务：Lab5 gdb调试系统调用以及返回

1. 基本梳理

1.1 两个GDB

- GDB 1: 调试QEMU进程本身（对应终端2）

- 用普通gdb;
- attach到qemu-system-riscv64的PID;
- 目的是看QEMU的源代码怎么跑。
- GDB 2: 调试QEMU里运行的uCore (对应终端3)
 - 用riscv64-unknown-elf-gdb;
 - target remote:1234;
 - 目的是控制 uCore 停在哪个指令、看寄存器/虚拟地址/触发某次访存。

1.2 三个终端

- 终端1: 跑QEMU (执行 `make debug` 启动uCore)。
- 终端2: 调QEMU源码。
- 终端3: 调uCore。

2. 调试过程

首先终端1执行 `make debug`。终端2执行 `pgrep -f qemu-system-riscv64` 得到进程ID (这里是8225)，接着执行 `sudo gdb`，然后执行 `attach 8225` 和 `handle SIGPIPE nostop noprint`，再执行 `continue` 就启动执行。终端3执行 `make gdb`，然后执行 `set remotetimeout unlimited` 和 `add-symbol-file obj/__user_exit.out`。

命令 `pgrep -f qemu-system-riscv64` 用于查找当前系统中正在运行的qemu-system-riscv64进程的PID (进程号)，方便attach来去调试QEMU本身。

命令 `handle SIGPIPE nostop noprint` 用于QEMU里有不少I/O/管道/socket行为，可能会触发SIGPIPE；默认GDB收到某些信号会停住并提示，导致看上去“程序卡了”。所以设置成收到SIGPIPE不停、不打印，让调试过程更顺滑。


```
问题 输出 调试控制台 终端 端口 1
jry@jry-VMware-Virtual-Platform:~/oslab/OSLab/lab5$ make debug
jry@jry-VMware-Virtual-Platform:~/oslab/qemu-4.1.1$ su
do gdb
warning: File "/home/jry/oslab/qemu-4.1.1/.gdbinit" au
to-loading has been declined by your `auto-load safe-p
ath' set to "$debugdir:$datadir/auto-load".
To enable execution of this file add
    add-auto-load-safe-path /home/jry/oslab/qemu-4
.1.1/.gdbinit
line to your configuration file "/root/.config/gdb/gdb
init".
To completely disable this security protection add
    set auto-load safe-path /
line to your configuration file "/root/.config/gdb/gdb
init".
For more information about this security protection se
e the
"Auto-loading safe path" section in the GDB manual. E
.g., run from the shell:
    info "(gdb)Auto-loading safe path"
(gdb) attach 8225
Attaching to process 8225
[New LWP 8227]
[New LWP 8226]
warning: could not find '.gnu_debugaltlink' file for /
lib/x86_64-linux-gnu/libglib-2.0.so.0
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu
/libthread_db.so.1".
0x00007f4ea911ba30 in __GI_ppoll (
    fds=0x63474bd86ab0, nfds=7,
    timeout=<optimized out>, sigmask=0x0)
    at ../sysdeps/unix/sysv/linux/ppoll.c:42
warning: 42  ../sysdeps/unix/sysv/linux/ppoll.c: 没
有那个文件或目录
(gdb) handle SIGPIPE nostop noprint
Signal      Stop      Print     Pass to program Descri
ption
SIGPIPE     No        No        Yes        Broken
pipe
(gdb) continue
Continuing.
jry@jry-VMware-Virtual-Platform:~/oslab/OSLab/lab5$ ma
ke gdb
add symbol table from file "obj/_user_exit.out"
(y or n) y
Reading symbols from obj/_user_exit.out...
(gdb) q
jry@jry-VMware-Virtual-Platform:~/oslab/OSLab/lab5$ ma
ke gdb
riscv64-unknown-elf-gdb \
-ex 'file bin/kernel' \
-ex 'set arch riscv:rv64' \
-ex 'target remote localhost:1234'
GNU gdb (SiFive GDB-Metal 10.1.0-2020.12.7) 10.1
Copyright (C) 2020 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu
.org/licenses/gpl.html>
This is free software: you are free to change and redi
stribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "--host=x86_64-linux-gnu --
target=riscv64-unknown-elf".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://github.com/sifive/freedom-tools/issues>.
Find the GDB manual and other documentation resources
online at:
<http://www.gnu.org/software/gdb/documentation/>.
For help, type "help".
Type "apropos word" to search for commands related to
"word".
Reading symbols from bin/kernel...
The target architecture is set to "riscv:rv64".
Remote debugging using localhost:1234
0x0000000000000100 in ?? ()
(gdb) set remotetimeout unlimited
(gdb) add-symbol-file obj/_user_exit.out
add symbol table from file "obj/_user_exit.out"
(y or n) y
Reading symbols from obj/_user_exit.out...
(gdb)
```

正式开始调试。

2.1 ecalls 指令

终端3执行 `b user/libs/syscall.c:26`，第26行正好是内联汇编的ecalls指令，但是我们发现断点实际上打在了第31行 `return ret;`，我猜测内联汇编可能是一个整体，只能将断点打在开头（19行，执行 `b user/libs/syscall.c:19`，这下对了），接着执行 `c`。

```
(gdb) b user/libs/syscall.c:26
Breakpoint 1 at 0x80010c: file user/libs/syscall.c, li
ne 31.
```

```
(gdb) i b
Num      Type           Disp Enb Address            Wha
t
1        breakpoint     keep y   0x00000000000080010c in
syscall at user/libs/syscall.c:31
(gdb) d 1
(gdb) i b
No breakpoints or watchpoints.
(gdb) b user/libs/syscall.c:19
Breakpoint 2 at 0x8000f8: file user/libs/syscall.c, li
ne 19.
```

```
问题 4 输出 调试控制台 终端 端口 1
jry@jry-Vmware-Virtual-Platform:~/oslab/OSLab/lab5$ make debug
Platform FWK1 features : RV64ACUIMBU
Platform Max HARTs : 8
Current Hart : 0
Firmware Base : 0x80000000
Firmware Size : 112 KB
Runtime SBI Version : 0.1

PMP0: 0x0000000080000000-0x000000008001ffff (A)
PMP1: 0x0000000000000000-0xffffffffffffff (A,R,W,X)
DTB Init
HartID: 0
DTB Address: 0x82200000
Physical Memory from DTB:
  Base: 0x0000000080000000
  Size: 0x0000000080000000 (128 MB)
  End: 0x0000000087ffffff
DTB init completed
(THU.CST) os is loading ...

Special kernel symbols:
  entry 0xc020004a (virtual)
  etext 0xc0205718 (virtual)
  edata 0xc02a62b8 (virtual)
  end 0xc02aa764 (virtual)
Kernel executable memory footprint: 682KB
memory management: default_pmm_manager
physical memory map:
  memory: 0x00000000, [0x80000000, 0x87ffffff].
vapaofset is 18446744070488326144
check_alloc_page() succeeded!
check_pgdir() succeeded!
check_boot_pgdir() succeeded!
use SLOB allocator
kmallocc_init() succeeded!
check_vma_struct() succeeded!
check_vmm() succeeded.
++ setup timer interrupts
kernel_execve: pid = 2, name = "forktest".
Breakpoint
[]

jry@jry-Vmware-Virtual-Platform:~/oslab/qemu-4.1.1$ su
do gdb
to-loading has been declined by your `auto-load safe-p
ath' set to "$debugdir:$datadir/auto-load".
To enable execution of this file add
  add-auto-load-safe-path /home/jry/oslab/qemu-4
.1.1/.gdbinit
line to your configuration file "/root/.config/gdb/gdb
init".
To completely disable this security protection add
  set auto-load safe-path /
line to your configuration file "/root/.config/gdb/gdb
init".
For more information about this security protection se
e the
"Auto-loading safe path" section in the GDB manual. E
.g., run from the shell:
  info "(gdb)Auto-loading safe path"
(gdb) attach 8225
Attaching to process 8225
[New LWP 8227]
[New LWP 8226]
warning: could not find '.gnu_debugaltlink' file for /
lib/x86_64-linux-gnu/libglib-2.0.so.0
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu
/libthread_db.so.1".
0x00007f4ea911ba30 in __GI_ppoll (
  fds=0x63474bd86ab0, nfd=7,
  timeout=<optimized out>, sigmask=0x0)
  at ../sysdeps/unix/sysv/linux/ppoll.c:42
warning: 42  ../sysdeps/unix/sysv/linux/ppoll.c: 没
有那个文件或目录
(gdb) handle SIGPIPE nostop noprnt
Signal Stop Print Pass to program Descri
ption
SIGPIPE No No Yes Broken
  pipe
(gdb) continue
Continuing.
(gdb) []

jry@jry-Vmware-Virtual-Platform:~/oslab/OSLab/lab5$ ma
ke gdb
For bug reporting instructions, please see:
<https://github.com/sifive/freedom-tools/issues>.
Find the GDB manual and other documentation resources
online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to
"word".
Reading symbols from bin/kernel...
The target architecture is set to "riscv:rv64".
Remote debugging using localhost:1234
0x0000000000001000 in ?? ()
(gdb) set remotetimeout unlimited
(gdb) add-symbol-file obj/_user_exit.out
add symbol table from file "obj/_user_exit.out"
(y or n) y
Reading symbols from obj/_user_exit.out...
(gdb) b user/libs/syscall.c:26
Breakpoint 1 at 0x80010c: file user/libs/syscall.c, li
ne 31.
(gdb) i b
Num Type Disp Enb Address Wha
t
1 breakpoint keep y 0x00000000000010c in
syscall at user/libs/syscall.c:31
(gdb) d 1
(gdb) i b
No breakpoints or watchpoints.
(gdb) b user/libs/syscall.c:19
Breakpoint 2 at 0x8000f8: file user/libs/syscall.c, li
ne 19.
(gdb) c
Continuing.

Breakpoint 2, syscall (num=2)
  at user/libs/syscall.c:19
19 asm volatile (
(gdb) []
```

终端3执行 `x/10i $pc` 查看接下来的汇编指令，再 `si`，直到`ecall`指令。

```
(gdb) x/10i $pc
=> 0x8000f8 <syscall+32>: ld a0,8(sp)
0x8000fa <syscall+34>: ld a1,40(sp)
0x8000fc <syscall+36>: ld a2,48(sp)
0x8000fe <syscall+38>: ld a3,56(sp)
0x800100 <syscall+40>: ld a4,64(sp)
0x800102 <syscall+42>: ld a5,72(sp)
0x800104 <syscall+44>: ecall
0x800108 <syscall+48>: sd a0,28(sp)
0x80010c <syscall+52>: lw a0,28(sp)
0x80010e <syscall+54>: addi sp,sp,144
(gdb) si
0x00000000008000fa 19 asm volatile (
(gdb)
0x00000000008000fc 19 asm volatile (
(gdb)
0x00000000008000fe 19 asm volatile (
(gdb)
0x0000000000800100 19 asm volatile (
(gdb)
0x0000000000800102 19 asm volatile (
(gdb)
0x0000000000800104 19 asm volatile (
(gdb) i r pc
pc 0x800104 0x800104 <syscall+44>
```

现在有一个非常现实的问题：终端2的断点该打在什么地方。开始问大模型：首先大模型的回答也比较含糊，说的是断点应当打在：“RISC-V 指令被翻译/执行 → 触发异常 → 进入异常处理 → 特权级切换”这条必经路径上（听君一席话，如听一席话）。但是它给出了两个关键函数：

`riscv_raise_exception()` 和 `riscv_cpu_do_interrupt()`。

你说得非常对：关键问题是QEMU源码里“应该断在哪儿”？请给出这个问题的解答。

好，这里我给你一个**“可以直接拿去用”的答案**。我会把**结论先给出**，再解释**为什么断在这些地方**，最后给你一套**最稳妥的断点打法（含兜底方案）**。

下面内容以 **QEMU 4.x 的 RISC-V TCG 实现为基准**，和你这份指导文档是**同一思路体系**。

一句话结论（你现在最想要的）

在 QEMU 源码中，调试系统调用（`ecall`）和返回（`sret`）时，断点应当打在：
“RISC-V 指令被翻译/执行 → 触发异常 → 进入异常处理 → 特权级切换”这条必经路径上。

具体到 QEMU (RISC-V) 源码，核心断点位置是这几类函数：

二、为什么这些断点是“必经之路”？

我们把一条 `ecall` 在 QEMU 中的生命周期串起来：



用IDE打开QEMU-4.1.1文件夹，搜索这两个函数，搜索到了之后又问大模型：哪个才是关键。这下得到了明确答复：`riscv_cpu_do_interrupt()` 函数才是关键。于是终端2执行 `b riscv_cpu_do_interrupt`。

请问op_helper.c中的riscv_raise_exception函数和cpu_helper.c中的riscv_cpu_do_interrupt函数的作用分别是什么？用户程序执行ecall时，哪一个处理中断的核心函数？

C cpu_helper.c

C op_helper.c

C cpu_helper.c:503

> 已使用 2 个引用

以下为两者的作用与流程简述：

- C op_helper.c 的 riscv_raise_exception
 - 作用：仅“提出/投递”异常。把异常号写入 cs->exception_index，然后通过 cpu_loop_exit_restore 抛回主循环，让主循环去真正处理。它不做任何特权级切换、CSR 写入或 PC 重定向。
 - 使用场景：同步异常（如 ecall、页故障、地址不对齐等）在译码/执行阶段调用该函数以触发异常投递。
- C cpu_helper.c 的 riscv_cpu_do_interrupt
 - 作用：真正“进入陷入/中断处理”的架构级实现。区分同步/异步（异常/中断）、根据 medeleg/mideleg 判定委派到 S/M 态；设置 mcause/scause、mepc/sepc、mbadaddr/sbadaddr (tval)、更新 mstatus 的 SIE/MIE、SPIE/MPLE、SPP/MPP 等 **明确解答** 特权级，并把 pc 跳转到 mtvec/stvec（含向量模式偏移）。
 - 使用场景：主循环在看到 cs->exception_index 后调用此函数完成陷入；硬件中断路径也会直接调用它完成处理。

用户程序执行 ecall 时，**处理中断/陷入的核心函数是 C cpu_helper.c 中的 riscv_cpu_do_interrupt。** riscv_raise_exception 只是把“异常投递出去”，真正完成寄存器更新与跳转的是 riscv_cpu_do_interrupt。

GPT-5 · 1x

再次启动，这下成功卡在了处理ecall指令的地方。

```
(gdb) c
Continuing.
[Switching to Thread 0x75b533fff6c0 (LWP 10329)]

Thread 2 "qemu-system-ris" hit Breakpoint 2, riscv_cpu_do_interrupt (cs=0x60d6f7b0d630)
    at /home/jry/oslab/qemu-4.1.1/target/riscv/cpu_helper.c:507
507      RISCVCPU *cpu = RISCV_CPU(cs);
(gdb) si

0x800110 <syscall+56>:      ret
0x800112 <sys_exit>:      mv      a1,a0
0x800114 <sys_exit+2>:      li      a0,1
0x800116 <sys_exit+4>:      j      0x8000d8 <syscall>
0x800118 <sys_fork>:      li      a0,2
0x80011a <sys_fork+2>:      j      0x8000d8 <syscall>
(gdb) si
```

那我们来看看 riscv_cpu_do_interrupt() 函数：

代码块

```
1 void riscv_cpu_do_interrupt(CPUState *cs)
2 {
3     #if !defined(CONFIG_USER_ONLY)
4
5         RISCVCPU *cpu = RISCV_CPU(cs);
6         CPURISCVState *env = &cpu->env;
7
8         /* cs->exception is 32-bits wide unlike mcause which is XLEN-bits wide
9          * so we mask off the MSB and separate into trap type and cause.
10        */
```

```

11     bool async = !(cs->exception_index & RISC_V_EXCP_INT_FLAG);
12     target_ulong cause = cs->exception_index & RISC_V_EXCP_INT_MASK;
13     target_ulong deleg = async ? env->mideleg : env->medeleg;
14     target_ulong tval = 0;
15
16     static const int ecall_cause_map[] = {
17         [PRV_U] = RISC_V_EXCP_U_ECALL,
18         [PRV_S] = RISC_V_EXCP_S_ECALL,
19         [PRV_H] = RISC_V_EXCP_H_ECALL,
20         [PRV_M] = RISC_V_EXCP_M_ECALL
21     };
22
23     if (!async) {
24         /* set tval to badaddr for traps with address information */
25         switch (cause) {
26             case RISC_V_EXCP_INST_ADDR_MIS:
27             case RISC_V_EXCP_INST_ACCESS_FAULT:
28             case RISC_V_EXCP_LOAD_ADDR_MIS:
29             case RISC_V_EXCP_STORE_AMO_ADDR_MIS:
30             case RISC_V_EXCP_LOAD_ACCESS_FAULT:
31             case RISC_V_EXCP_STORE_AMO_ACCESS_FAULT:
32             case RISC_V_EXCP_INST_PAGE_FAULT:
33             case RISC_V_EXCP_LOAD_PAGE_FAULT:
34             case RISC_V_EXCP_STORE_PAGE_FAULT:
35                 tval = env->badaddr;
36                 break;
37             default:
38                 break;
39         }
40         /* ecall is dispatched as one cause so translate based on mode */
41         if (cause == RISC_V_EXCP_U_ECALL) {
42             assert(env->priv <= 3);
43             cause = ecall_cause_map[env->priv];
44         }
45     }
46
47     trace_riscv_trap(env->mhartid, async, cause, env->pc, tval, cause < 16 ?
48         (async ? riscv_intr_names : riscv_excp_names)[cause] : "(unknown)");
49
50     if (env->priv <= PRV_S &&
51         cause < TARGET_LONG_BITS && ((deleg >> cause) & 1)) {
52         /* handle the trap in S-mode */
53         target_ulong s = env->mstatus;
54         s = set_field(s, MSTATUS_SPIE, env->priv_ver >= PRIV_VERSION_1_10_0 ?
55             get_field(s, MSTATUS_SIE) : get_field(s, MSTATUS_UIE << env-
>priv));
56         s = set_field(s, MSTATUS_SPP, env->priv);

```

```

57     s = set_field(s, MSTATUS_SIE, 0);
58     env->mstatus = s;
59     env->scause = cause | (((target_ulong)async << (TARGET_LONG_BITS - 1));
60     env->sepc = env->pc;
61     env->sbadaddr = tval;
62     env->pc = (env->stvec >> 2 << 2) +
63         ((async && (env->stvec & 3) == 1) ? cause * 4 : 0);
64     riscv_cpu_set_mode(env, PRV_S);
65 } else {
66     /* handle the trap in M-mode */
67     target_ulong s = env->mstatus;
68     s = set_field(s, MSTATUS_MPIE, env->priv_ver >= PRIV_VERSION_1_10_0 ?
69         get_field(s, MSTATUS_MIE) : get_field(s, MSTATUS_UIE << env-
>priv));
70     s = set_field(s, MSTATUS_MPP, env->priv);
71     s = set_field(s, MSTATUS_MIE, 0);
72     env->mstatus = s;
73     env->mcause = cause | ~(((target_ulong)-1) >> async);
74     env->mepc = env->pc;
75     env->mbadaddr = tval;
76     env->pc = (env->mtvec >> 2 << 2) +
77         ((async && (env->mtvec & 3) == 1) ? cause * 4 : 0);
78     riscv_cpu_set_mode(env, PRV_M);
79 }
80
81 /* NOTE: it is not necessary to yield load reservations here. It is only
82  * necessary for an SC from "another hart" to cause a load reservation
83  * to be yielded. Refer to the memory consistency model section of the
84  * RISC-V ISA Specification.
85  */
86
87 #endif
88     cs->exception_index = EXCP_NONE; /* mark handled to qemu */
89 }

```

- 读取当前异常号 `cs->exception_index`，判断是否为异步中断（带 `RISCV_EXCP_INT_FLAG`）还是同步异常。
- 提取 `cause`（原因码）并选择委派寄存器：异步用 `mideleg`，同步用 `medeleg`。
- 若为同步异常，针对带地址信息的异常，从 `env->badaddr` 填充 `tval`；若是 `ecall`，则根据当前特权级把通用的 `U_ECALL` 映射为 `U/S/H/M` 对应的 `ecall cause`。
- 打印 `trace` 信息（用于调试）。
- 判断是否委派到 S 态处理：若当前特权 $\leq S$ ，且对应 `cause` 在委派位图中置位，则走 S 态陷入路径：

- 更新 mstatus: $SPIE \leftarrow SIE$ (或 $UIE \ll \text{priv}$) , $SPP \leftarrow$ 当前特权, SIE 清零。
 - 写 scause (含异步标志位于最高位)、 $sepc \leftarrow$ 当前 pc、 $sbadaddr \leftarrow tval$ 。
 - 设置 pc 跳转到 stvec 基址, 若向量模式且为异步中断则加上 $cause * 4$ 偏移。
 - 切换特权级到 S。
- 否则在 M 态处理:
 - 更新 mstatus: $MPIE \leftarrow MIE$ (或 $UIE \ll \text{priv}$) , $MPP \leftarrow$ 当前特权, MIE 清零。
 - 写 mcause (最高位标识异步)、 $mepc \leftarrow$ 当前 pc、 $mbadaddr \leftarrow tval$ 。
 - 设置 pc 跳转到 mtvec 基址, 向量模式下异步中断加 $cause * 4$ 偏移。
 - 切换特权级到 M。
- 最后将 `cs->exception_index` 清为 `EXCP_NONE`, 表示该次陷入已处理。

2.2 sret 指令

继续跑。首先找到 `sret` 指令在 `kern/trap/trapentry.S` 第 140 行, 终端 3 执行 `b kern/trap/trapentry.S:141` 和 `c`。又问大模型, 接下来终端 2 的断点打在哪里, 经过一番折腾终于知道了应该打在 `helper_sret()` 函数。

你刚刚说得非常好, 我断点打在了 `riscv_cpu_do_interrupt` 函数, 成功看到了 `ecall` 指令的处理过程。现在我想看 `sret` 指令的处理过程, 调试 QEMU 进程本身的终端的断点应该打在哪里?

很好, 你已经把 `ecall → riscv_cpu_do_interrupt()` 这条线抓住了; 那 `sret` 这边的“对称点”就是: **QEMU 在执行 `sret` 时做的“特权返回”处理函数。**

在“调 QEMU 进程本身”的那个终端里, 断点建议打在下面这几类位置 (按稳定性从高到低给你)。

1) 最推荐: 直接断在“sret 的 helper / 实现函数”上

在 QEMU 的 RISC-V TCG 里, `sret` 通常走的是 **helper** 路径 (而不是 `do_interrupt`), 名字在不同版本可能略有差异, 所以最稳的方法是用 **正则断点兜底**:

在 *attach QEMU 的宿主机 gdb* 里 (终端 2) :

终端 2 执行 `b helper_sret`, 继续跑, 成功卡在了处理 `sret` 指令的地方。

```

(gdb) b helper_sret
Breakpoint 3 at 0x60d6f5a31405: file /home/jry/oslab/
qemu-4.1.1/target/riscv/op_helper.c, line 76.
(gdb) c
Continuing.
[Switching to Thread 0x75b533fff6c0 (LWP 10329)]

Thread 2 "qemu-system-ris" hit Breakpoint 3, helper_s
ret (env=0x60d6f7b16040,
    cpu_pc_deb=18446744072637910818)
    at /home/jry/oslab/qemu-4.1.1/target/riscv/op_hel
per.c:76
76      if (!(env->priv >= PRV_S)) {
(gdb)
0xfffffffffc0200f2e <kernel_execve_ret+4>:
    ld  s1,280(a0)
0xfffffffffc0200f32 <kernel_execve_ret+8>:
    sd  s1,280(a1)
0xfffffffffc0200f36 <kernel_execve_ret+12>:
    ld  s1,272(a0)
0xfffffffffc0200f3a <kernel_execve_ret+16>:
    sd  s1,272(a1)
0xfffffffffc0200f3e <kernel_execve_ret+20>:
    ld  s1,264(a0)
0xfffffffffc0200f42 <kernel_execve_ret+24>:
    sd  s1,264(a1)
(gdb) ni

```

那我们来还是看看 `helper_sret()` 函数：

代码块

```

1  target_ulong helper_sret(CPURISCVState *env, target_ulong cpu_pc_deb)
2  {
3      if (!(env->priv >= PRV_S)) {
4          riscv_raise_exception(env, RISCV_EXCP_ILLEGAL_INST, GETPC());
5      }
6
7      target_ulong retpc = env->sepc;
8      if (!riscv_has_ext(env, RVC) && (retpc & 0x3)) {
9          riscv_raise_exception(env, RISCV_EXCP_INST_ADDR_MIS, GETPC());
10     }
11
12     if (env->priv_ver >= PRIV_VERSION_1_10_0 &&
13         get_field(env->mstatus, MSTATUS_TSR)) {
14         riscv_raise_exception(env, RISCV_EXCP_ILLEGAL_INST, GETPC());
15     }
16
17     target_ulong mstatus = env->mstatus;
18     target_ulong prev_priv = get_field(mstatus, MSTATUS_SPP);
19     mstatus = set_field(mstatus,
20         env->priv_ver >= PRIV_VERSION_1_10_0 ?
21         MSTATUS_SIE : MSTATUS_UIE << prev_priv,
22         get_field(mstatus, MSTATUS_SPIE));
23     mstatus = set_field(mstatus, MSTATUS_SPIE, 0);
24     mstatus = set_field(mstatus, MSTATUS_SPP, PRV_U);
25     riscv_cpu_set_mode(env, prev_priv);
26     env->mstatus = mstatus;
27
28     return retpc;
29 }

```

- 访问权限检查：若当前特权级 < S（Supervisor），抛出非法指令异常。

- 计算返回地址：读取 env->sepc 作为 retpc，并在不支持 RVC（压缩指令）时检查 retpc 是否按 4 字节对齐，不对齐则抛出取指地址不对齐异常。
- TSR 检查：在 priv 1.10.0 及以上，若 mstatus.TSR 置位，则禁止执行 sret，抛出非法指令异常。
- 恢复中断使能位：
 - 读取 mstatus，prev_priv = SPP（保存的先前特权级）。
 - 将 SIE 恢复为 SPIE（老版本通过 UIE<<prev_priv），并清零 SPIE。
 - 将 SPP 设为 U（规范要求返回时清空为 U）。
- 切换特权级：调用 riscv_cpu_set_mode(env, prev_priv) 将 CPU 模式切回先前特权级。
- 写回 mstatus 并返回 retpc：返回地址由上层用来更新 PC，完成从陷入返回。

2.3 指令翻译（TCG Translation）

TCG Translation 是 QEMU 在运行时对 guest 架构指令进行动态翻译的过程。QEMU 并不直接执行 guest 指令，而是**先将其翻译为与宿主机无关的 TCG 中间表示（IR）**，再由 TCG 将 IR 编译为宿主机可执行的代码。对于如 ecall、sret 等涉及特权级和异常的复杂指令，翻译阶段会生成对 helper 函数的调用，由 helper 在执行阶段完整模拟硬件语义。

QEMU 的执行路径：

代码块

```

1    【1】 指令翻译（Translation）
2        RISC-V 指令 → TCG IR
3        |
4    【2】 代码生成（TCG）
5        TCG IR → x86_64 机器码
6        |
7    【3】 执行（Execution）
8        x86_64 CPU 执行生成的代码

```

至此，基本完成了lab5的分支任务：GDB调试系统调用以及返回。